

CPP Library Blueprints for Python Bindings

Rohit Goswami · 2023-06-28 · design · bindings · python · cpp

Design guidelines for thin Python wrappers to C++ libraries

Background

I recently ended up writing and rewriting a series of libraries to essentially get them to a point where I could write bindings to them. In doing so, some thoughts on design have evolved into this post.

False starts

Originally, I had the library and bindings in one repository, which was easier to hack on, and had fewer moving parts. I rashly assumed that using branches would provide a modicum of reproducibility. As I quickly found though, this got out of hand when the code needed to be refactored to suit the bindings and wasn't always backported to `main` in a timely fashion

Setup Preliminaries

We will be as general in our suggestions as is humanely possible, but when we need to reach for specific examples, they will be in context of the `potlib` project ([library](#), [bindings](#)), which is small and innocuous enough to grok hopefully.

Tools

For most new projects, I use `pybind11` [^fn:1] with `meson`.

- `pdm` is used for dependencies / scripting / environments
- `mesonpy` is the backend
- `cibuildwheel` is used for actually getting the wheels for `pypi`
 - This is a bit of a nightmare, need to use [repology](#) and fiddle around with figuring out which version of each OS is used by each specification
 - Any kind of system based package management was exceedingly slow, so sticking to `pip` and `meson` managed dependencies is the safest bet

Library Layout

Taking `potlib` as a concrete example, my current favored layout looks like this:

```

CppCore
├─ examples
│   └─ calling_cuh2.cc
│   └─ calling_ljpot.cc
├─ gtests
│   └─ CuH2PotTest.cc
├─ meson.build
└─ src
    ├─ base_types.cc
    ├─ base_types.hpp
    ├─ CuH2
    ├─ helpers.cc
    ├─ helpers.hpp
    ├─ LennardJones
    ├─ Potential.cc
    ├─ Potential.hpp
    └─ pot_types.hpp

```

Linking to the library

[Git submodules](#) are generally the first thing to reach for when working with repositories which depend on each other, but I found `meson` [subprojects](#) to be less finicky if the library and bindings are all compatible with `meson`. The documentation omits setting up a `.wrap` file for a project, so consider the following (from `pypotlib`):

```

[wrap-git]
directory=potlib
url=https://github.com/TheochemUI/potlib.git
revision=5d029b9

[provide]
potlib=git

```

Which can be placed in `subprojects/potlib.wrap` of the bindings repo and should be committed.

Miscellaneous `meson` points

Personally I gravitate towards having a few standard default variables defined at the top level:

```

_args = [] # Extra arguments
_deps = [] # Dependencies
_linkto = [] # All the sub-libraries
_incdirs = [] # All the includes

```

Along with the `clang` and `gcc` on Unix related dependencies:

```

# Dependencies
# libm for Unix systems
m_dep = cppc.find_library('m', required: false)
_deps += m_dep
# For clang
_deps += [declare_dependency(link_args: '-lstdc++')]

```

This makes it easier to pull variables from subprojects in a transparent manner: Linking to the library itself is best done by declaring a dependency `[^fn:2]`:

```
# ----- Library Dependencies
potlib_proj = subproject('potlib', version: '0.1')
# optionally use:
# default_options: ['default_library=static'])
potlib_dep = declare_dependency(link_with: potlib_proj.get_variable('_linkto'),
                                dependencies: potlib_proj.get_variable('_deps'))
_deps += [ potlib_dep ]
```

Setting up targets is then also pretty straightforward:

```
blahlib = library('blah',
                  src_files,
                  dependencies: _deps,
                  cpp_args: _args,
                  link_with: _linkto,
                  install: true)
```

Binding Conventions

- Always use `clang-format`.
- All strings must be explicitly identified `" "s`
 - ****not for doc-strings**** in `pybind11`

General

- Always use `namespace py = pybind11;`
- For `Blah.cpp` the binding will be `pybinds/py_blah.cc`
- Most header content must be in `pybinds/py_wrapper.hpp`
 - The exceptions are base classes like `py_potential.hpp` since these need to be imported by child files like `potentials/py_morse.cc`
 - If a header is supplied, then the corresponding `.cc` should **only** reference the header, which in turn should declare other dependencies.
 - This only applies to files at the same tree/folder level
- Every method must have:
 - arguments defined
 - documentation string

Example:

```
.def("setCell", &Matter::setCell, "Sets the cell dimensions"s, py::arg("AtomMatrix newCell"))
```

Naming

Naming is hard. To simplify long filenames and deep hierarchies, the following conventions are established:

- Within `objectivefunctions`, `ObjectiveFunction` can be replaced by `objfunc`
 - So `MatterObjectiveFunction` is bound in `py_matterobjfunc.cc`

Class boilerplate

```
void py_objectName(py::module_ &m) {
    py::class_<ObjectName>(m, "ObjectName", py::dynamic_attr()) // dynamic incurs a penalty
    /*
    ** Constructors
    */

    /*
    ** Operators
    */

    /*
    ** Methods
    */

    /*
    ** Parameters
    */

    /*
    ** Python helpers
    */

    .def("__repr__", [](const ObjectName &a) { return "<ObjectName object>"; });
}
```

Function arguments

- First one must be `py::arg()`
- Subsequent arguments may be `""_a`
- When there is a non-obvious type used in the arguments, document it
 - e.g. `AtomMatrix` and `VectorXd`
 - Use `_` for spaces, e.g. `AtomMatrix_pos` for arguments `(AtomMatrix pos)`
- When using overloads, always note the actual arguments as comments

```
py::overload_cast<long /*nAtoms*/,
                AtomMatrix /*positions*/,
                VectorXi /*atomicNrs*/,
                double * /*energy*/,
                Matrix3d /*box*/,
                int /*nImages*/
                >(&Potential::force),
```

Structuring Bindings

Although a single file can be used for the `pybind11` bindings, I prefer a more structured approach, consider the layout below:

```

├─ pyb11_srcs
│   └─ CuH2
│       ├── py_cuh2pot.cc
│       └─ py_cuh2pot.hpp
│   └─ LennardJones
│       ├── py_ljpot.cc
│       └─ py_ljpot.hpp
│   ├── py_potential.cc
│   ├── py_potential.hpp
│   ├── py_pottypes.cc
│   ├── py_wrapper.cc
│   └─ py_wrapper.hpp

```

Which mirrors the library design.

- Add to `py_wrapper.hpp`

```

PYBIND11_MODULE(eonclient, m) {
    ...
    py_newthing(m);
}

```

- Make `py_newthing.cc`

```

// clang-format off
#include "py_wrapper.hpp"
// Binding code (unless in py_wrapper)
#include "../newthing.h"
// clang-format on

void py_newthing(py::module_ &m) {
    /* Wrapping details here */
}

```

For a concrete example consider this [pypotlib commit](#).

Conclusions

So far this covered creating a single compiled extension for thin bindings to an existing `c++` library. Using PRs and submodules turned out to be insufficient to maintain code quality and I ended up shifting to a subproject based workflow. Most of these evolved during my work on [pypotlib](#), and a newer version of `e0N`, the long timescale dynamics code, which isn't (as of April 2024) public yet^[fn:3]. A follow up post will cover adding pure python code which can be used for generating a more pythonic interface and also the requirements for distribution via PyPI.